2026 IEEE International Conference on Artificial Intelligence (CAI)
8-10 May, 2026. Granada, Spain

Agentic AI as a Cybersecurity Attack Surface: Threats, Exploits, and Defenses in Runtime Supply Chains

Xiaochong Jiang[1,†], Shiqi Yang[2,†], Wenting Yang[3,†], Yichen Liu[4], Cheng Ji[5]

2026 IEEE Conference on Artificial Intelligence (CAI) | 979-8-3315-6039-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/CAI68641.2026.11536564

***Abstract*— Agentic systems built on large language models (LLMs) extend beyond text generation to autonomously retrieve information and invoke tools. This runtime execution model shifts the attack surface from build-time artifacts to inference-time dependencies, exposing agents to manipulation through untrusted data and probabilistic capability resolution. While prior work has focused on model-level vulnerabilities, security risks emerging from cyclic and interdependent runtime behavior remain fragmented.**

We systematize these risks within a unified runtime framework, categorizing threats into data supply chain attacks (transient context injection and persistent memory poisoning) and tool supply chain attacks (discovery, implementation, and invocation). We further identify the *Viral Agent Loop*, in which agents act as vectors for self-propagating generative worms without exploiting code-level flaws. Finally, we advocate a Zero-Trust Runtime Architecture that treats context as untrusted control flow and constrains tool execution through cryptographic provenance rather than semantic inference.

***Index Terms*— Agentic AI, Cybersecurity of AI Systems, Runtime Attacks, Tool-Integrated LLMs, Indirect Prompt Injection**

I. Introduction

Large Language Models (LLMs) are increasingly deployed as autonomous agentic systems, extending beyond passive text generation. Modern agents retrieve external information, maintain persistent memory, and invoke tools capable of modifying digital or physical states [1], [2]. Consequently, agent behavior is shaped not only by model parameters and developer prompts, but also by dynamically acquired data and capabilities during execution.

This execution model introduces a distinct security paradigm, which we term *Stochastic Dependency Resolution*. Unlike traditional software, where dependencies are resolved and fixed prior to deployment (e.g., `import numpy` mapping to a specific binary hash), agentic systems assemble their effective execution context at runtime through probabilistic semantic decisions. Retrieved documents, external APIs, and tools thus become implicit inference-time dependencies, transforming context from passive input into an active component of the attack surface.

Such dynamics challenge conventional cybersecurity assumptions. Autonomous agents routinely process untrusted data and execute privileged actions, often without human mediation. As a result, adversaries need not compromise infrastructure or model weights; instead, they can manipulate environmental artifacts—data sources, memory stores, or tool interfaces—to influence downstream behavior.

Prior work has examined model-level and framework-level vulnerabilities, including training data poisoning, backdoors, and supply-chain risks [3]. More recent studies identify runtime threats such as indirect prompt injection during tool orchestration [4], tool stream manipulation [5], and workflow auditing weaknesses [6]. However, these efforts typically analyze isolated failure modes rather than the structural properties of agentic execution.

As agents become increasingly autonomous, this fragmented view proves insufficient. Runtime perception, memory updates, and tool invocation are interdependent, and agent outputs may re-enter the system as future inputs, enabling cyclic and persistent failure modes. Addressing these risks requires a unified framework that treats inference-time dependency resolution as a primary cybersecurity concern.

This paper synthesizes existing research on runtime data and tool supply chain threats in agentic systems. By organizing prior work according to structural patterns of agent execution, we clarify relationships among diverse attack techniques and highlight systemic security challenges inherent to autonomous AI.

II. Background and Threat Assumptions

Autonomous agentic systems differ from traditional software primarily in how they acquire and trust dependencies. Conventional applications rely on a *static software supply chain*, where libraries and binaries are explicitly declared, resolved, and verified prior to deployment. In contrast, agentic systems continuously assemble their effective execution context during inference. Information sources, retrieved documents, and external tools—often selected autonomously—become implicit runtime dependencies that directly influence both reasoning and action.

This shift fundamentally alters the security boundary. Compromise may occur without modifying source code,

†These authors contributed equally to this work.
[1]Xiaochong Jiang is an Independent Researcher, Seattle, WA, USA jiang.xiaoc@northeastern.edu
[2]Shiqi Yang is an Independent Researcher, New York City, NY, USA sy3506@nyu.edu
[3]Wenting Yang is an Independent Researcher, San Diego, CA, USA wey023@ucsd.edu
[4]Yichen Liu is an Independent Researcher, Seattle, WA, USA yil160@ucsd.edu
[5]Cheng Ji is an Independent Researcher, Champaign, IL, USA chengji5@illinois.edu
© 2026 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 18:27:15 UTC from IEEE Xplore. Restrictions apply.

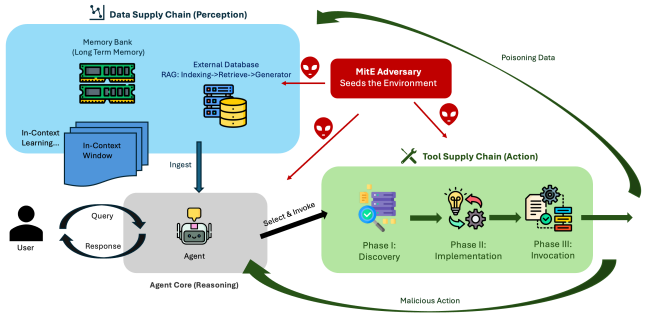


Fig. 1. The Agentic Runtime Supply Loop and associated attack surfaces. The **Man-in-the-Environment (MitE)** adversary targets two dynamic dependencies: (1) **The Data Supply Chain (Left):** Adversaries inject malicious content into the Context Window, Memory Bank, or External Databases, altering agent perception. (2) **The Tool Supply Chain (Right):** Adversaries subvert capability resolution across Phase I (Discovery), Phase II (Implementation), and Phase III (Invocation). Crucially, the loop closes: poisoned perception triggers unauthorized tool actions, while tool outputs re-enter the environment as tainted context, enabling persistent compromise and viral-style propagation.

model weights, or infrastructure, but purely through manipulation of the external environments that agents observe and interact with.

A. Static vs. Dynamic Supply Chains

In traditional software systems, the supply chain consists of a finite and enumerable set of artifacts—source code, libraries, and binaries—resolved at build or deployment time. These dependencies are explicitly declared, sourced from identifiable vendors, and amenable to pre-execution verification through signatures, audits, or vulnerability scanning. As a result, security mechanisms primarily focus on protecting infrastructure and validating static artifacts.

Agentic systems operate under a fundamentally different model. Their dependencies are acquired at runtime and may include natural-language documents, retrieved knowledge, APIs, or executable tools. Which dependencies are selected depends on the agent's reasoning process and the evolving state of its environment, making the resulting supply chain dynamic, open-ended, and difficult to enumerate in advance. Moreover, the effective payload of an attack is often semantic rather than binary, relying on language-based manipulation of the model's inference rather than memory corruption or control-flow hijacking.

Table I summarizes the key distinctions between static and dynamic supply chains.

B. Threat Assumptions and Adversarial Setting

We consider adversaries who do not compromise model weights, system prompts, or hosting infrastructure. Instead, the adversary leverages the **Prompt-Data Isomorphism**—the phenomenon where retrieved passive data (D) is upcast into active instructions (I) by the LLM's reasoning engine. We refer to this threat model as a **Man-in-the-Environment (MitE)** adversary. The environment itself acts as the hostile actor, embedding malicious content into data sources or capabilities that agents execute voluntarily.

TABLE I
COMPARISON: STATIC VS. DYNAMIC SUPPLY CHAINS

Feature	Static (Traditional)	Dynamic (Agentic)
Resolution Time	Build / Deploy Time	Inference / Runtime
Dependency Type	Libraries, Binaries	Data Context, APIs, Tools
Topology	Directed Acyclic	Cyclic (Feedback Loops)
Attack Surface	Code Vulnerabilities	Semantic Manipulation
Upstream Source	Verified Vendor	Runtime Information Sources

Throughout this paper, we refer to such adversarial sources as *runtime suppliers*, emphasizing the supply-chain role played by the MitE adversary.

Analogous to a malicious package maintainer in a traditional software ecosystem, a runtime supplier publishes data or capabilities that appear benign but are designed to influence agent behavior when consumed. Unlike direct prompt-based attacks initiated by users, these threats are indirect: the agent voluntarily retrieves and integrates the malicious artifact as part of normal execution.

C. Adversary Capabilities

We assume that a runtime supplier may influence agent behavior along two primary dimensions.

First, the adversary may inject malicious content into data sources that the agent is likely to retrieve, such as web pages, documents, or comments. When incorporated into the agent's context, such content can steer reasoning through indirect prompt injection or related forms of semantic manipulation.

Second, the adversary may publish deceptive tools or API wrappers in public registries. By anticipating the agent's tool-selection logic, these tools can masquerade as legitimate functionality while performing unintended actions, such as data exfiltration or unauthorized operations, once invoked.

D. Adversary Goals

These capabilities enable several classes of undesirable outcomes. An adversary may induce *stealthy misalignment*, causing the agent to deviate from its intended objective without triggering safety mechanisms. They may compromise confidentiality by inducing the agent to leak contextual information during execution. In more severe cases, agent actions may propagate malicious content back into the environment, allowing attacks to persist or amplify as outputs are re-ingested as future inputs.

III. THE DATA SUPPLY CHAIN

This section explores agent's **Perception Module**, the component of an agentic system responsible for acquiring, parsing, and normalizing external information from the environment including instruction prompts, historical dialogs, external knowledge database, etc. The attacker can inject malicious instructions or data into external information sources to subvert the agent's subsequent reasoning and action. We categorize data-driven attacks based on where control is enforced, rather than on the surface form of the payload.

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 18:27:15 UTC from IEEE Xplore. Restrictions apply.

Agents increasingly rely on retrieval-augmented generation (RAG) to incorporate external knowledge at inference time. In practice, such systems are structured as information retrieval (IR) pipelines that dynamically fetch and rank external artifacts during execution [7]. Retrieved documents therefore become implicit runtime dependencies rather than passive inputs.

As with standard LLM supply chain risks, agentic architectures share a fundamental vulnerability: the **context window**. This window serves as a "shared space" where secure system commands and untrusted external data are concatenated, inadvertently leveraging the model's **In-Context Learning** (ICL) capability to override pre-trained safety alignments.

However, the severity of these attacks is amplified in agents through expanded **scope** and **persistence**. While traditional LLM data risks primarily result in "one-off" semantic corruption (e.g., misinformation or toxicity), agent-specific exploits target the **action-perception loop**. In an agentic workflow, the data supply chain is not merely a passive input stream; it functions as a functional **control signal** for tool-use and long-term planning. Consequently, a single poisoned data point does not yield simply a "bad answer"—it induces a compromised operational state. This manifests itself through a shift from **static** to **dynamic** vulnerability: while standard LLM risks are often transient and limited to a single response, agentic risks can persist across sessions by prompt infection [8], poisoning the agent's self-updating memory [9] or triggering unauthorized high-stakes execution of external tools [10].

In this section, we categorize these data attacks based on the target's persistence: within-session manipulation and across-session manipulation.

A. Within-Session Manipulation

Within-Session Manipulation is transient and typically expire once the session is reset. This category targets the agent's **Contextual Window**, which is the immediate buffer of active tokens used for in-the-moment reasoning.

1) **Indirect Prompt Injection:** The malicious payload is covertly embedded in a third-party, untrusted data source that the agent is instructed to process [11]. The **mechanism** relies on the agent's **tool use** (e.g., a web browser or email API), which fetches the untrusted contents and prepends it to the user's query Q before passing the entire context to the LLM. Furthermore, this vector extends beyond textual data into multimodal domains: as demonstrated in [12], adversaries can embed subtle adversarial perturbations into *images or sounds* that, upon processing, are decoded into a textual payload that steers the model to an attacker-chosen output. Recent studies show that such human-imperceptible perturbations can lead to a 98% success rate in steering model outputs [12].
2) **In-Context Learning**: These attacks exploit the mechanism of **In-Context Learning**(ICL) [13], where the model adjusts behavior based on patterns in the immediate history. In the paper of **Many-Shot Jailbreaking** [14], the attacker floods the short-term memory with a large volume of fictitious dialogues depicting the agent helpfully answering malicious queries. Due to the attention mechanism's prioritization of ICL over pre-trained safety guardrails, the agent "learns" from this poisoned short-term context that compliance with harmful requests is the correct behavior for the current session. Research indicates that as the shot count increases, the ASR follows a power-law growth, effectively rendering standard alignment filters obsolete within a single session.

B. Across-Session Manipulation

This vector targets the integrity of the agent's accumulated knowledge and historical context stored in **External Memory**. These attacks are persistent, allowing a single injection to influence the agent across multiple future sessions.

1) **Knowledge Base Contamination:** During **Retrieval Aggregated Generation (RAG) retrieval** stage, the agent performs a semantic similarity search to extract relevant snippets D' from the vectorized corpus, which are then integrated into the prompt to provide the LLM with grounded evidence. In the **generation** phase, the reasoning engine utilizes In-Context Learning to synthesize these external fragments into a coherent response or plan P', a process that inadvertently treats the retrieved D' as a high-priority instruction for the execution of the current task. For example, Zhong et al. [15] showed that attackers can generate adversarial strings that hijack the vector space to ensure that they are retrieved for a wide range of indiscriminate user questions. Building on this, *PoisonedRAG* injects strategically decomposed texts into a retrieval-optimized trigger and a generation-inducing payload. Studies show that poisoning only 0.1% of the external corpus can result in 70% ASR for targeted queries [9]. Approaches such as *as AGENTPOISON* [10] utilize constrained optimization to embed a stealthy backdoor. These backdoors remain dormant until a specific trigger in a future query activates the retrieval of the poisoned trace, with reported ASRs exceeding 80% in various autonomous tasks [10]. Furthermore, recent work on graph-structured retrieval systems demonstrates that even advanced GraphRAG pipelines remain vulnerable to relation-level poisoning attacks, where adversaries inject malicious edges into the knowledge graph to influence multi-hop reasoning [16].
2) **Long Term Memory Poisoning:** Beyond static retrieval, another critical attack surface is the agent's **Long-Term Memory (LTM)**. Unlike the transient context window, LTM serves as a persistent repository that allows agents to store and retrieve episodic experiences (past trajectories) and semantic knowledge across different sessions [17]. This read-write autonomy enables the agent to accumulate "experience," but

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 18:27:15 UTC from IEEE Xplore. Restrictions apply.

it also creates a feedback loop vulnerability where the agent can be tricked into "poisoning itself." Without having the privilege to the memory bank, *MINJA* framework [18] proposed an attack mechanism that can inject malicious records into the memory bank only via queries. It appends an instruction-heavy 'indication prompt' to a benign query containing a specific 'victim term' to trigger the injection, achieving an average ASRs 76.8%.

IV. The Tool Supply Chain

While the Data Supply Chain (Section III) governs what information an agent incorporates into its internal state, the Tool Supply Chain governs what the agent can *do* in the external environment. It binds natural-language intent to executable capabilities whose effects extend beyond the model boundary, directly modifying files, networks, accounts, or physical systems.

Failures in this chain therefore affect more than epistemic correctness. By corrupting the binding between **intent**, **capability**, and **authority**, an adversary can cause an agent to exercise unintended or excessive privileges while maintaining apparently coherent reasoning traces. We characterize such failures as **capability hijacking**, rather than purely semantic manipulation.

a) Tool Abstraction and Security Invariants.: We model a tool as a delegated operational capability that binds semantic intent to executable code under bounded authority. Concretely, a tool is defined by its identifier, semantics, interface, authority scope, side effects, and output contract. Secure tool use depends on four core invariants: **identity integrity** (the resolved `Tool_ID` refers to the intended implementation), **semantic binding** (the selected tool matches user intent), **authority bounding** (execution obeys least-privilege constraints), and **implementation integrity** (the loaded code matches the authenticated tool). Violation of any single invariant is sufficient to induce externally observable harm.

b) Pipeline Decomposition.: The Tool Supply Chain consists of three sequential capability-binding phases. Phase I (**Discovery**) resolves intent to a concrete `Tool_ID`. Phase II (**Implementation**) fetches and instantiates the corresponding code. Phase III (**Invocation**) executes the tool under a specific authority scope and side-effect envelope. Attacks may target any phase by corrupting tool identity, implementation integrity, or authority binding. Phase III operates closest to the execution boundary, governing logical control over permissions, credentials, and side effects even when the correct tool and implementation are selected.

Multi-tool LLM agents have been proposed for structured workflows that coordinate evidence retrieval and tool-based validation [19]. Such architectures illustrate how dynamic tool orchestration is becoming a standard design pattern, further amplifying the security implications of runtime capability binding.

A. Phase I: Attacks on Discovery (Intent → `Tool_ID`)

The Discovery phase resolves natural-language intent into a concrete tool identifier prior to any code execution. In the benign setting, tool resolution follows:

$$f_{\text{resolve}}(\text{Intent}) \rightarrow \texttt{Tool_ID} \tag{1}$$

Under adversarial conditions, this resolution process may be perturbed by semantic noise or optimized manipulation of the surrounding context:

$$f_{\text{resolve}}(\text{Intent} + \epsilon_{\text{adv}}) \rightarrow \texttt{Tool_ID}_{\text{malicious}} \tag{2}$$

where ϵ_{adv} represents adversarially crafted semantic perturbations injected into tool descriptions, metadata, or retrieval context. Attacks at this stage compromise *selection* rather than execution, manipulating how tools are indexed, described, or retrieved so that the agent voluntarily selects an adversary-controlled capability.

1) **Hallucination Squatting:** During planning, agents may hallucinate plausible but non-existent tool identifiers. Adversaries can preemptively register these identifiers as malicious packages, transforming a would-be resolution error into successful execution of attacker-controlled code. Empirical evidence shows that hallucinated package names are predictable and recur across models and prompts, enabling systematic squatting on high-probability "ghost" identifiers [20].
2) **Semantic Masquerading:** Discovery mechanisms commonly rely on semantic similarity between user intent and tool descriptions. By adversarially crafting metadata to maximize textual overlap with common queries, attackers can manipulate retrieval rankings and displace legitimate tools. This breaks semantic binding by associating a correct intent with an incorrect capability. Benchmarks show that even minor perturbations to tool descriptions can significantly degrade selection accuracy for strong models [21], and tool-selection studies explicitly identify description noise as a practical misdirection channel [22].

B. Phase II: Attacks on Implementation (Load `Tool_ID` → Runtime Code)

After a tool identifier is resolved, the Implementation phase fetches and instantiates executable logic:

$$\text{Load}(\texttt{Tool_ID}) \rightarrow \text{Runtime Code}. \tag{3}$$

Unlike Discovery attacks, which manipulate selection, Implementation attacks preserve correct tool choice while corrupting the code that executes.

1) **Hidden Backdoors and Malicious Extensions:** A tool may implement its advertised functionality under benign conditions while embedding trigger-based logic that activates malicious behavior in specific contexts. Such backdoors allow agents to maintain high apparent utility while silently executing unintended actions, compromising implementation integrity without visible deviation in reasoning traces. This failure mode has

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 18:27:15 UTC from IEEE Xplore. Restrictions apply.

been demonstrated in backdoored agent backends that achieve high attack success rates while preserving normal task performance [23].

2) **Transitive Dependency Exploitation:** Implementation integrity may also be compromised indirectly through dependency resolution. Tool setup often triggers installation of auxiliary packages, and common package managers permit arbitrary code execution during installation. In autonomous workflows, agents routinely modify their environments to complete tasks, treating dependency installation as a functional step rather than a security-sensitive operation [24]. As a result, selecting a benign tool may implicitly execute malicious transitive dependencies before the intended tool is ever invoked.

C. Phase III: Attacks on the Invocation Boundary

The Invocation phase governs how a selected tool is executed at runtime. Unlike earlier phases, these attacks arise from failures in constraining *how* a tool is used rather than which tool is selected or what code is loaded.

We model invocation as:

$$\text{Invoke}(T, \theta, \mathcal{C}) \rightarrow (o, \Delta), \tag{4}$$

where θ denotes invocation arguments and $\mathcal{C}$ the execution context (e.g., credentials and accessible resources). Phase III attacks aim to induce unauthorized or unintended side effects Δ, or to expose sensitive elements of $\mathcal{C}$, even when T is legitimate.

1) **Over-Privileged Invocation:**
Agents are frequently provisioned with broad permissions and long-lived credentials to ensure task completion. When invocation lacks capability-based constraints, adversaries can induce a confused-deputy–like failure in which the agent applies its authority to actions not justified by the originating intent.
This phenomenon has been formalized in recent work on privilege escalation in LLM-based multi-agent systems, where natural-language communication channels allow low-privilege components to trigger high-privilege tool execution [25]. Because authority usage is driven by the agent's internal reasoning rather than explicit human approval, such failures are non-interactive and may be difficult to detect or reverse once committed.
Empirical studies further show that indirect prompt injection can blur the boundary between data and instructions, enabling authority hijacking in tool-integrated agents [11]. This threat is operationalized in *InjecAgent*, where agents can be induced to perform unauthorized transfers or data exfiltration when invocation is not mediated by explicit permission checks [26]. Additional evaluations demonstrate that agents frequently misjudge side-effect boundaries even for syntactically valid tool calls, resulting in severe consequences such as irreversible data deletion [27].

2) **Argument Injection and Logical Boundary Breaches.**
Even when a tool T is correctly selected and invoked under valid credentials, adversarial manipulation of invocation parameters θ can induce side effects that exceed intended task semantics. In this setting, the capability itself is legitimate, but the mapping between arguments and permissible state transitions is insufficiently constrained.
This failure mode is analogous to application-layer exploits such as Server-Side Request Forgery (SSRF), where a trusted intermediary relays attacker-controlled inputs into a privileged execution context. Empirical evidence shows that indirect prompt injection can reliably steer downstream API arguments without altering tool identity [28], and recent work demonstrates how prompt injection increasingly merges with classical web vulnerabilities to exploit application-layer weaknesses [29]. Industry analyses further identify Cross-Plugin Request Forgery (CPRF), where injected content causes agents to silently generate malicious parameters for connected tools.
Unlike over-privileged invocation, which violates *authority bounding*, argument injection violates *semantic constraint binding* at the invocation boundary. From a supply-chain perspective, the tool and authority remain authentic, yet logical boundary enforcement between intent, argument space, and external side effects fails.

V. The Viral Agent: Closing the Loop

The preceding sections analyze the Data and Tool supply chains largely in isolation, where compromise remains localized to a single agent or execution step. However, the defining departure of the Agentic Supply Chain from traditional software lifecycles lies not merely in new attack vectors, but in the possibility of *autonomous propagation*. When the Data Supply Chain (Discovery) is coupled with the Tool Supply Chain (Invocation), agent outputs may re-enter the environment as future inputs, closing the loop between consumption and contamination. This transforms isolated compromise into a recursive, ecosystem-level failure mode.

A. The Self-Replicating Cycle

We define a **Viral Agent Loop** as a recursive scenario in which an agent Ag_A processes a payload P that induces a side effect Δ via tool invocation, such that Δ is later discovered and incorporated as input by another agent Ag_B.

Building on the Phase III invocation semantics introduced earlier, this process can be formalized as:

$$\text{Invoke}(\text{Ag}_A, P) \xrightarrow{\text{writes}} \Delta_{\text{env}} \ \wedge \ \text{Retrieve}(\text{Ag}_B) \ni \Delta_{\text{env}} \supseteq P \tag{5}$$

Here, the invocation side effect Δ_{env} becomes part of the effective context $\mathcal{C}$ for Ag_B. This transformation turns the agent from a passive victim into an active carrier. Unlike isolated invocation failures (Section IV), where harm is bounded to a single execution, the Viral Agent Loop

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 18:27:15 UTC from IEEE Xplore. Restrictions apply.

introduces temporal recursion: Δ_{env} persists as discoverable state and may trigger downstream agents without further adversarial interaction. Unlike traditional malware, which relies on code execution vulnerabilities, agentic propagation exploits the model's instruction-following behavior under otherwise legitimate tool permissions.

B. Propagation Vectors and Empirical Evidence

1) Generative Worms: The feasibility of self-replicating agentic attacks has been empirically demonstrated. Cohen et al. [30] introduce Morris II, showing that adversarial prompts can self-replicate across GenAI agents. An agent receiving a poisoned input is induced to exfiltrate context and propagate the payload to downstream agents via authorized communication tools. This mirrors the 1988 Morris Worm but operates entirely at the semantic layer; patching underlying code cannot mitigate the threat, only aligning the semantic reasoning process can.

2) Persistent Ecosystem Contamination: A more persistent vector arises when agents possess write access to shared knowledge stores. An agent manipulated via Data Supply Chain poisoning may upload compromised artifacts—such as documents or code—back into repositories including corporate wikis or version-control systems.

This induces a feedback loop in which poisoned outputs are re-ingested through retrieval mechanisms, allowing the attack to persist across sessions and agents. In effect, agents become *producers and consumers* of malicious content, progressively degrading the integrity of the shared knowledge base.

C. Topological Shift: From Pipeline to Cycle

Importantly, the Viral Agent Loop is not a new attack primitive but a structural property of cyclic agent ecosystems. These dynamics imply a fundamental topological shift in supply chain security assumptions. Traditional software supply chains are well-modeled as **Directed Acyclic Graphs (DAGs)**, where artifacts flow unidirectionally from upstream producers to downstream consumers and compromise eventually terminates.

In contrast, the Agentic Supply Chain forms a **cyclic graph**. Because invocation side effects Δ may re-enter the system as discoverable inputs, the distinction between supplier and consumer collapses. Security mechanisms that assume acyclic dependency resolution may therefore fail once agent outputs are permitted to influence future execution contexts.

From a defensive standpoint, agentic systems must be analyzed as dynamic, self-referential systems rather than static pipelines.

VI. Systematization of Existing Defenses

Prior work emphasizes static LLM alignment [3]. However, dynamic Agentic Supply Chains require a *Zero Trust Runtime* architecture. We systematize existing defenses across five runtime layers (Perception, Memory, Resolution, Implementation, Invocation), mapped to our threat model in Table II.

TABLE II
Systematization of Existing Agentic Supply Chain Defenses

Supply Chain Component	Targeted Attack Vector	Existing Defensive Paradigms
Data (Perception)	Direct & Indirect Prompt Injection	Instruction Hierarchy [31], Intent Verification [32], [33]
Data (Memory)	RAG & Agent-State Poisoning	Statistical Filtering [34], Audited Memory Writes [35]
Tool (Phase I)	Hallucination Squatting	Static Registry Allowlists
Tool (Phase II)	Dependency Pollution	Signed SBOMs, SLSA Frameworks [36]
Tool (Phase III)	Privilege Escalation	Scoped Permissions (e.g., MCP [37]), Multi-stage Arbiters [38]

While the paradigms in Table II offer valuable localized mitigations, they fundamentally inherit assumptions from the *Static Model Supply Chain*. By evaluating these defenses through their respective supply chain components, we expose why they remain structurally insufficient against the MitE adversary.

A. Securing the Data Supply Chain

Defenses at this layer must (i) structurally separate operator instructions from passive data, and (ii) police memory integrity.

a) Perception Layer (Instruction & Intent): Frameworks such as the **Instruction Hierarchy** [31] and **Intent Verification** [32], [33] encapsulate untrusted external data within rigid structural delimiters. However, this approach underestimates the *epistemic uncertainty* inherent in In-Context Learning. Because agents continuously upcast passive data into active context, adversaries can exploit the Prompt-Data Isomorphism to bypass syntactic delimiters using purely semantic payloads. Consequently, generative worms [30] can still induce malicious behavior without violating the prescribed structural hierarchy.

b) Memory Layer (Persistence): To prevent persistent knowledge corruption, mechanisms rely on **Statistical Filtering** [34] or **Audited Memory Writes** [35]. Unfortunately, empirical evidence demonstrates that statistical anomaly detection falls short against structural graph poisoning [16]. When an adversary subtly manipulates relational edges in an agent's long-term memory or external RAG corpus, the retrieved text appears statistically benign but logically steers the agent into a compromised operational state.

B. Securing the Tool Supply Chain

Tool defenses operate sequentially across the execution pipeline to restore identity, integrity, and authority.

a) Resolution and Implementation (Phases I & II): To counter hallucination squatting and dependency pollution, current proposals advocate for **Static Registry Allowlists** [20] and verifiable provenance via **SBOMs** [36]. While these measures guarantee identity and implementation integrity, they are blind to the agent's semantic intent. An

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 18:27:15 UTC from IEEE Xplore. Restrictions apply.

agent manipulated by MitE will simply utilize a cryptographically verified, perfectly legitimate tool to execute an unauthorized, malicious action.

b) Invocation and Semantic Firewalls (Phase III): Recognizing that standard OS-level permissions cannot detect logical boundary breaches, recent architectures introduce **Semantic Firewalls** (e.g., Vigil [5], MCP-Guard [38]) and ephemeral capabilities (e.g., MCP [37]). These multi-stage arbiters utilize a secondary LLM to audit the primary worker's tool calls. Yet, this "AI-guarding-AI" paradigm suffers from the exact same fundamental vulnerability: the arbiter model must process the identical tainted context to evaluate the action. Thus, the arbiter inherits the same *stochastic capabilities* and vulnerability to prompt injection, failing to bridge the translation gap between benign user intent and hazardous execution.

c) Phase III: Invocation Integrity & Semantic Firewalls: This phase represents the final line of defense before side effects occur. Since standard permission models cannot detect semantic misuse (e.g., a legally authorized but contextually malicious file deletion), we argue for a **Defense-in-Depth** approach combining AI oversight with structural bounding:

C. The Illusion of Acyclic Defense

Collectively, these component-level defenses treat agentic execution as an acyclic pipeline. They apply static patches to dynamic vulnerabilities, completely failing to neutralize the *Viral Agent Loop* (Section V) where output side effects re-enter as tainted input. Securing the ecosystem against cyclic propagation demands abandoning heuristic patches in favor of the *Zero-Trust Runtime Architecture* proposed in Section VII.

VII. The Zero-Trust Agentic Runtime Architecture

While the defenses systematized in Section VI offer valuable point-mitigations, they suffer from a fundamental limitation: they treat epistemic uncertainty and stochastic capabilities as isolated machine learning safety issues. Addressing the **Man-in-the-Environment (MitE)** adversary requires recognizing that these are interconnected supply chain vulnerabilities.

We argue that the security paradigm must shift entirely from the *Static Model Supply Chain* (which secures training data and static weights) to the *Dynamic Agentic Runtime Supply Chain* (which secures real-time data consumption, tool execution, and cyclic propagation). To achieve this, we propose the transition to a **Zero-Trust Runtime Architecture**, predicated on three core architectural imperatives.

A. Deterministic Capability Binding

The reliance on LLMs to probabilistically "guess" tool identifiers via semantic similarity is structurally insecure and constitutes the root cause of Phase I and Phase II tool supply chain attacks. A Zero-Trust Runtime must enforce **Deterministic Capability Binding**. We advocate for the deployment of **Cryptographically Bound Registries** that verify the semantic integrity of tool providers. Under this architecture, tool execution is bounded strictly by cryptographic provenance rather than semantic likelihood. The translation from natural-language intent to executable capability must be mediated by a verified resolution engine that mathematically guarantees identity integrity, eliminating the "Hallucination Gap" entirely.

B. Neuro-Symbolic Information Flow Control

Current static analysis techniques are insufficient for dynamic agents because they fail to account for the *Viral Agent Loop*, where outputs re-enter the environment as tainted context. To prevent self-propagating agentic worms, we propose **Runtime Taint Analysis** engineered for neural systems.

In this framework, untrusted external inputs (e.g., retrieved web text) are strictly tagged as `TAINTED`. The runtime must track this taint through the non-deterministic transformations of the LLM's reasoning chain. If a tainted reasoning trace attempts to invoke a write-privileged sink (e.g., `send_email` or `git_push`), the execution is blocked pending explicit sanitization or human-in-the-loop approval. Because neural networks can wash standard metadata tags through paraphrasing, this flow control must be underpinned by **Cryptographic Provenance Ledgers** [39], ensuring an immutable record of data lineage from perception to action.

C. The Auditor-Worker Architecture (Semantic Firewalls)

Standard OS-level sandboxing cannot prevent logical boundary breaches (Phase III attacks) because the runtime cannot distinguish between a legally authorized but contextually malicious action (e.g., deleting a critical database file at the behest of an injected prompt) and a benign task. A single model cannot simultaneously optimize for both stochastic helpfulness and deterministic safety.

We propose the **Auditor-Worker Architecture**. This paradigm structurally decouples execution from oversight by placing an isolated **Supervisor Model** as an inline **Semantic Firewall**. Operating via speculative execution (extending concepts from preliminary frameworks like *Vigil* [5], *AgentGuard* [6], and multi-stage cognitive arbiters like *MCP-Guard* [38]), the Supervisor analyzes the proposed tool call, its parameters, and the lineage of the prompt *before* the action is committed to the environment. This hard structural split bridges the translation gap between benign user intent and hazardous execution, enforcing least-privilege constraints at the semantic layer.

VIII. Conclusion and Future Directions

This Systematization of Knowledge formalizes the topological shift from static software security to the *Dynamic Agentic Runtime Supply Chain (DRSC)*. We defined the *Man-in-the-Environment (MitE)* adversary, who exploits *Prompt-Data Isomorphism* to weaponize an agent's perception and tool usage. Crucially, we identified the *Viral Agent Loop*,

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 18:27:15 UTC from IEEE Xplore. Restrictions apply.

where agents become vectors for self-propagating generative worms without underlying code vulnerabilities.

To secure the dynamic agentic runtime against the Man-in-the-Environment (MitE) adversary, future research must address three interconnected imperatives. First, we require **Realistic Evaluation Benchmarks** that move beyond static simulations with closed-world assumptions and episodic amnesia [26], [40]. Future frameworks must evaluate how agents dynamically locate and trust tools in open-world, hostile environments to capture persistent memory poisoning and cyclic viral propagation. Second, the development of **Robust Capability Protocols** is essential; while standardization efforts like MCP [37] are promising, their reliance on semantic resolution leaves them vulnerable to description injection [41], necessitating mathematically verifiable, Name-Squatting Resilient Registries. Finally, mitigating epistemic uncertainty requires **Neuro-Symbolic Taint Tracking**—developing reliable mechanisms to maintain cryptographic provenance tags across the non-deterministic, stochastic abstractions of transformer layers, even when syntactic representations change.

Ultimately, in the Agentic Supply Chain, context *is* code. Until security architectures fully embrace this prompt-data isomorphism, autonomous agents will remain fundamentally vulnerable operators.

AI USE DISCLOSURE

The authors acknowledge the use of Gemini to refine the clarity and grammar of the text. The authors reviewed and revised the output and take full responsibility for the content of this article.

REFERENCES

[1] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," 2023.

[2] Z. Xi *et al.*, "The rise and potential of large language model based agents: A survey," 2023.

[3] S. Wang, Y. Zhao, X. Hou, and H. Wang, "Large language model supply chain: A research agenda," 2024.

[4] H. An, J. Zhang, T. Du, C. Zhou, Q. Li, T. Lin, and S. Ji, "Ipiguard: A novel tool dependency graph-based defense against indirect prompt injection in llm agents," 2025.

[5] J. Lin, Z. Zhou, Z. Zheng, S. Liu, T. Xu, Y. Chen, and E. Chen, "Vigil: Defending llm agents against tool stream injection via verify-before-commit," 2026.

[6] J. Chen and S. L. Cong, "Agentguard: Repurposing agentic orchestrator for safety evaluation of tool orchestration," 2025.

[7] N. Arabzadeh *et al.*, "Ir-rag@ sigir25: The second edition of the workshop on information retrieval's role in rag systems," in *Proc. of SIGIR*, 2025, pp. 4168–4171.

[8] D. Lee and M. Tiwari, "Prompt infection: Llm-to-llm prompt injection within multi-agent systems," *arXiv preprint arXiv:2410.07283*, 2024.

[9] W. Zou, R. Geng, B. Wang, and J. Jia, "Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models," 2024.

[10] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases," 2024.

[11] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," 2023.

[12] E. Bagdasaryan, T.-Y. Hsieh, B. Nassi, and V. Shmatikov, "Abusing images and sounds for indirect instruction injection in multi-modal LLMs," 2023.

[13] Q. Dong *et al.*, "A survey on in-context learning," 2024.

[14] Anil *et al.*, "Many-shot jailbreaking," in *Proc. of NeurIPS*, 2024. [Online]. Available: https://www.anthropic.com/research/many-shot-jailbreaking

[15] Z. Zhong, Z. Huang, A. Wettig, and D. Chen, "Poisoning retrieval corpora by injecting adversarial passages," 2023.

[16] J. Liang, Y. Wang, C. Li, R. Zhu, T. Jiang, N. Gong, and T. Wang, "Graphrag under fire," 2025.

[17] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen, "A survey on the memory mechanism of large language model based agents," 2024.

[18] S. Dong, S. Xu, P. He, Y. Li, J. Tang, T. Liu, H. Liu, and Z. Xiang, "Memory injection attacks on llm agents via query-only interaction," 2026.

[19] Z. Cui, T. Huang, C.-E. Chiang, and C. Du, "Toward verifiable misinformation detection: A multi-tool llm agent framework," in *Proceedings of the 2025 International Conference on Generative Artificial Intelligence for Business*, 2025, pp. 179–185.

[20] J. Spracklen *et al.*, "We have a package for you! a comprehensive analysis of package hallucinations by code generating llms," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, arXiv:2406.10279.

[21] J. Ye *et al.*, "Rotbench: A multi-level benchmark for evaluating the robustness of large language models in tool learning," in *Proc. of EMNLP*, 2024, pp. 313–333.

[22] J. Y. others, "Toolsword: Unveiling safety issues of large language models in tool learning across three stages," 2024.

[23] Y. Wang, D. Xue, S. Zhang, and S. Qian, "Badagent: Inserting and activating backdoor attacks in llm agents," 2024.

[24] R. Fang, R. Bindu, A. Gupta, Q. Zhan, and D. Kang, "Llm agents can autonomously hack websites," 2024.

[25] Z. Ji, D. Wu, W. Jiang, P. Ma, Z. Li, Y. Gao, S. Wang, and Y. Li, "Taming various privilege escalation in llm-based agent systems: A mandatory access control framework," *arXiv preprint arXiv:2601.11893*, 2026.

[26] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents," 2024.

[27] Y. Ruan *et al.*, "Toolemu: Identifying the risks of LM agents with an LM-emulated sandbox," in *Proc. of ICLR*, 2024.

[28] N. K.-S. Kong, "Injectbench: An indirect prompt injection benchmarking framework," Ph.D. dissertation, Virginia Tech, 2024.

[29] J. McHugh, K. Šekrst, and J. Cefalu, "Prompt injection 2.0: hybrid ai threats," *arXiv preprint arXiv:2507.13169*, 2025.

[30] S. Cohen, R. Bitton, and B. Nassi, "Here comes the ai worm: Unleashing zero-click worms that target genai-powered applications," 2025.

[31] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training llms to prioritize privileged instructions," 2024.

[32] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The task shield: Enforcing task alignment to defend against indirect prompt injection in llm agents," 2024.

[33] M. Kang, C. Xiang, S. Kariyappa, C. Xiao, B. Li, and E. Suh, "Mitigating indirect prompt injection via instruction-following intent analysis," 2025.

[34] M. Kim, H. Lee, and H. Koo, "Rescuing the unpoisoned: Efficient defense against knowledge corruption attacks on rag systems," 2025.

[35] Q. Wei, T. Yang, Y. Wang, X. Li, L. Li, Z. Yin, Y. Zhan, T. Holz, Z. Lin, and X. Wang, "A-memguard: A proactive defense framework for llm-based agent memory," 2025.

[36] OpenSSF, "Supply-chain levels for software artifacts (slsa) v1.0," 2024, https://slsa.dev/.

[37] Anthropic, "The model context protocol (mcp): Standardizing ai context," 2024, https://www.anthropic.com/news/model-context-protocol.

[38] W. Xing *et al.*, "Mcp-guard: A multi-stage defense-in-depth framework for securing model context protocol in agentic ai," 2026.

[39] T. A. Syed, M. A. Almutairi, and M. A. Moaty, "Toward trustworthy agentic ai: A multimodal framework for preventing prompt injection attacks," 2025.

[40] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents," 2024.

[41] Z. Wang, R. Zhang, Y. Liu, W. Fan, W. Jiang, Q. Zhao, H. Li, and G. Xu, "Mpma: Preference manipulation attack against model context protocol," 2025.

Authorized licensed use limited to: TU Bergakademie Freiberg. Downloaded on October 05,2026 at 18:27:15 UTC from IEEE Xplore. Restrictions apply.